| Ex. No. 10 | Working with MongoDB using Shell Commands |
|---|---|
| **Date of Exercise** | 02.11.2026 |
| **GitHub Host Link** | https://praneettigga.github.io/web-tech-lab/ |

## Aim

To demonstrate database creation, document insertion, querying, updating, indexing, and deletion in a document-oriented NoSQL database using MongoDB Shell (mongosh) commands.

## Description

MongoDB is a popular NoSQL document database designed for high availability, performance, and dynamic schema scalability. Unlike traditional Relational Database Management Systems (RDBMS) that organize data into fixed tables and rows, MongoDB stores data in flexible, JSON-like structures called BSON (Binary JSON) documents grouped inside collections.

The MongoDB Shell (mongosh) is an interactive JavaScript interface used to interact with MongoDB instances. It enables database administrators and developers to manipulate data, manage database indexes, and execute administrative functions directly via command-line interface.

**Database & Collection Management:** Switching active databases (use) and instantiating collections (createCollection). A database is created lazily the first time data is stored in it.

**Create Operations:** Adding single or multiple BSON documents into collections using insertOne() and insertMany(). Each document automatically receives a unique _id field.

**Read Operations:** Searching documents using exact matching, conditional comparison operators ($gte, $lt), logical operators ($or), array matching, projections, and document counts.

**Update Operations:** Modifying existing field values ($set), incrementing numerical counts ($inc), or appending values to array fields ($push) using updateOne() and updateMany().

**Indexing:** Building unique single-field indexes (createIndex()) to optimize query lookup performance and to reject duplicate values.

**Delete Operations:** Removing specific documents matching filter constraints via deleteOne() and deleteMany(), and removing a whole collection or database with drop() and dropDatabase().

## Procedure:

1. Install MongoDB Community Server and the MongoDB Shell (mongosh), and start the mongod server.

2. Open a terminal and connect to the server by running mongosh.

3. Run show dbs to list the databases, use universityDB to switch to (and create) the database, db to confirm it, createCollection("students") to create the collection, and show collections to list it.

4. Insert one document with insertOne() and three more with insertMany() into the students collection.

5. Query the data with find(), an exact filter, the $gte operator, the $or operator, a projection, findOne() and countDocuments().

6. Modify the data with updateOne() using $set, updateMany() using $push, and updateOne() using $inc, and check the changed documents.

7. Create a unique index on student_id with createIndex() and list the indexes with getIndexes().

8. Delete documents with deleteOne() and deleteMany(), and check the remaining documents.

9. Clean up by dropping the collection with drop() and the database with dropDatabase().

10. Save all the commands in mongodb-commands.js, then push the code to GitHub and host it using GitHub Pages to obtain the deployment link.

## Program

### mongodb-commands.js

```
// ============================================================================
// EXPERIMENT: WORKING WITH MONGODB USING SHELL COMMANDS (mongosh)
// ============================================================================

// ----------------------------------------------------------------------------
// 1. DATABASE & COLLECTION MANAGEMENT
// ----------------------------------------------------------------------------

// List all existing databases on the MongoDB server
show dbs;

// Switch context to (or create if non-existent) database 'universityDB'
use universityDB;

// Confirm current active database context
db;

// Create an explicit collection named 'students'
db.createCollection("students");

// Display all collections in current database
show collections;



// ----------------------------------------------------------------------------
// 2. CREATE OPERATIONS (INSERTION)
// ----------------------------------------------------------------------------

// Insert a single document into 'students' collection
db.students.insertOne({
  student_id: 101,
  name: "Alice Smith",
  age: 20,
  major: "Computer Science",
  gpa: 3.8,
  skills: ["Python", "MongoDB"],
  enrollmentDate: new Date("2023-09-01")
});

// Insert multiple documents into 'students' collection
db.students.insertMany([
  {
    student_id: 102,
```

```
    name: "Bob Jones",
    age: 22,
    major: "Data Science",
    gpa: 3.4,
    skills: ["R", "Python", "SQL"],
    enrollmentDate: new Date("2022-09-01")
  },
  {
    student_id: 103,
    name: "Charlie Brown",
    age: 21,
    major: "Computer Science",
    gpa: 2.8,
    skills: ["Java", "C++"],
    enrollmentDate: new Date("2023-01-15")
  },
  {
    student_id: 104,
    name: "Diana Prince",
    age: 23,
    major: "Cybersecurity",
    gpa: 3.9,
    skills: ["Linux", "Python", "Networking"],
    enrollmentDate: new Date("2021-09-01")
  }
]);


// ---------------------------------------------------------------------------
// 3. READ OPERATIONS (QUERYING)
// ---------------------------------------------------------------------------

// Retrieve all documents in the collection
db.students.find();

// Retrieve documents matching exact filter criteria
db.students.find({ major: "Computer Science" });

// Query using comparison operator ($gte: Greater than or equal to)
db.students.find({ gpa: { $gte: 3.5 } });

// Query using logical operator ($or) and array value check
db.students.find({
  $or: [
    { major: "Cybersecurity" },
    { skills: "MongoDB" }
  ]
});

// Projection query: select specific fields (1 to include, 0 to exclude)
db.students.find({}, { name: 1, gpa: 1, major: 1, _id: 0 });

// Fetch a single document matching criteria
db.students.findOne({ student_id: 101 });

// Count total documents matching a specified condition
db.students.countDocuments({ major: "Computer Science" });


// ---------------------------------------------------------------------------
// 4. UPDATE OPERATIONS
// ---------------------------------------------------------------------------
```

```
// Update a single document using $set operator
db.students.updateOne(
  { student_id: 101 },
  { $set: { gpa: 3.9, age: 21 } }
);

// Append an element to an array field across multiple documents using $push
db.students.updateMany(
  { major: "Computer Science" },
  { $push: { skills: "Git" } }
);

// Increment a numeric value by 1 using $inc operator
db.students.updateOne(
  { student_id: 102 },
  { $inc: { age: 1 } }
);



// ------------------------------------------------------------------------
// 5. INDEXING
// ------------------------------------------------------------------------

// Create a unique index on student_id to enforce uniqueness and optimize lookups
db.students.createIndex({ student_id: 1 }, { unique: true });

// List all indexes configured on the collection
db.students.getIndexes();



// ------------------------------------------------------------------------
// 6. DELETE OPERATIONS & CLEANUP
// ------------------------------------------------------------------------

// Delete a single document matching specified filter criteria
db.students.deleteOne({ student_id: 103 });

// Delete all documents matching a condition (e.g., GPA below 3.0)
db.students.deleteMany({ gpa: { $lt: 3.0 } });

// Drop the collection from the database
db.students.drop();

// Drop the current active database
db.dropDatabase();
```

# Output

## Database and collection management




*Figure 1: Listing databases, switching to universityDB and creating the students collection*

## Create operations

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.insertOne({
    student_id: 101,
    name: "Alice Smith",
    age: 20,
    major: "Computer Science",
    gpa: 3.8,
    skills: ["Python", "MongoDB"],
    enrollmentDate: new Date("2023-09-01")
  });
{
  acknowledged: true,
  insertedId: ObjectId('6af1fa4df15443c6333ffd82')
}
universityDB> db.students.insertMany([
    {
      student_id: 102,
      name: "Bob Jones",
      age: 22,
      major: "Data Science",
      gpa: 3.4,
      skills: ["R", "Python", "SQL"],
      enrollmentDate: new Date("2022-09-01")
    },
    {
      student_id: 103,
      name: "Charlie Brown",
      age: 21,
      major: "Computer Science",
      gpa: 2.8,
      skills: ["Java", "C++"],
      enrollmentDate: new Date("2023-01-15")
    },
    {
      student_id: 104,
      name: "Diana Prince",
      age: 23,
      major: "Cybersecurity",
      gpa: 3.9,
      skills: ["Linux", "Python", "Networking"],
      enrollmentDate: new Date("2021-09-01")
    }
  ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6af1fa4df15443c6333ffd83'),
    '1': ObjectId('6af1fa4df15443c6333ffd84'),
    '2': ObjectId('6af1fa4df15443c6333ffd85')
  }
}
universityDB>
```

*Figure 2: insertOne() and insertMany() acknowledging the four inserted documents*

## Read operations (1)

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.find();
[
  {
    _id: ObjectId('6af1fa4df15443c6333ffd82'),
    student_id: 101,
    name: 'Alice Smith',
    age: 20,
    major: 'Computer Science',
    gpa: 3.8,
    skills: [ 'Python', 'MongoDB' ],
    enrollmentDate: ISODate('2023-09-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('6af1fa4df15443c6333ffd83'),
    student_id: 102,
    name: 'Bob Jones',
    age: 22,
    major: 'Data Science',
    gpa: 3.4,
    skills: [ 'R', 'Python', 'SQL' ],
    enrollmentDate: ISODate('2022-09-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('6af1fa4df15443c6333ffd84'),
    student_id: 103,
    name: 'Charlie Brown',
    age: 21,
    major: 'Computer Science',
    gpa: 2.8,
    skills: [ 'Java', 'C++' ],
    enrollmentDate: ISODate('2023-01-15T00:00:00.000Z')
  },
  {
    _id: ObjectId('6af1fa4df15443c6333ffd85'),
    student_id: 104,
    name: 'Diana Prince',
    age: 23,
    major: 'Cybersecurity',
    gpa: 3.9,
    skills: [ 'Linux', 'Python', 'Networking' ],
    enrollmentDate: ISODate('2021-09-01T00:00:00.000Z')
  }
]
universityDB> db.students.find({ major: "Computer Science" });
[
  {
    _id: ObjectId('6af1fa4df15443c6333ffd82'),
    student_id: 101,
    name: 'Alice Smith',
    age: 20,
    major: 'Computer Science',
    gpa: 3.8,
    skills: [ 'Python', 'MongoDB' ],
    enrollmentDate: ISODate('2023-09-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('6af1fa4df15443c6333ffd84'),
    student_id: 103,
    name: 'Charlie Brown',
    age: 21,
    major: 'Computer Science',
    gpa: 2.8,
    skills: [ 'Java', 'C++' ],
    enrollmentDate: ISODate('2023-01-15T00:00:00.000Z')
  }
]
universityDB>
```

*Figure 3: find() with no filter and find() with an exact match on major*

## Read operations (2)

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.find({ gpa: { $gte: 3.5 } });
[
  {
    _id: ObjectId('6af1fa4df15443c6333ffd82'),
    student_id: 101,
    name: 'Alice Smith',
    age: 20,
    major: 'Computer Science',
    gpa: 3.8,
    skills: [ 'Python', 'MongoDB' ],
    enrollmentDate: ISODate('2023-09-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('6af1fa4df15443c6333ffd85'),
    student_id: 104,
    name: 'Diana Prince',
    age: 23,
    major: 'Cybersecurity',
    gpa: 3.9,
    skills: [ 'Linux', 'Python', 'Networking' ],
    enrollmentDate: ISODate('2021-09-01T00:00:00.000Z')
  }
]
universityDB> db.students.find({
    $or: [
      { major: "Cybersecurity" },
      { skills: "MongoDB" }
    ]
  });
[
  {
    _id: ObjectId('6af1fa4df15443c6333ffd82'),
    student_id: 101,
    name: 'Alice Smith',
    age: 20,
    major: 'Computer Science',
    gpa: 3.8,
    skills: [ 'Python', 'MongoDB' ],
    enrollmentDate: ISODate('2023-09-01T00:00:00.000Z')
  },
  {
    _id: ObjectId('6af1fa4df15443c6333ffd85'),
    student_id: 104,
    name: 'Diana Prince',
    age: 23,
    major: 'Cybersecurity',
    gpa: 3.9,
    skills: [ 'Linux', 'Python', 'Networking' ],
    enrollmentDate: ISODate('2021-09-01T00:00:00.000Z')
  }
]
universityDB>
```

*Figure 4: find() with the $gte comparison operator and the $or logical operator*

## Read operations (3)

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.find({}, { name: 1, gpa: 1, major: 1, _id: 0 });
[
  { name: 'Alice Smith', major: 'Computer Science', gpa: 3.8 },
  { name: 'Bob Jones', major: 'Data Science', gpa: 3.4 },
  { name: 'Charlie Brown', major: 'Computer Science', gpa: 2.8 },
  { name: 'Diana Prince', major: 'Cybersecurity', gpa: 3.9 }
]
universityDB> db.students.findOne({ student_id: 101 });
{
  _id: ObjectId('6af1fa4df15443c6333ffd82'),
  student_id: 101,
  name: 'Alice Smith',
  age: 20,
  major: 'Computer Science',
  gpa: 3.8,
  skills: [ 'Python', 'MongoDB' ],
  enrollmentDate: ISODate('2023-09-01T00:00:00.000Z')
}
universityDB> db.students.countDocuments({ major: "Computer Science" });
2
universityDB>
```

*Figure 5: Projection, findOne() and countDocuments()*

## Update operations

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.updateOne(
    { student_id: 101 },
    { $set: { gpa: 3.9, age: 21 } }
  );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
universityDB> db.students.updateMany(
    { major: "Computer Science" },
    { $push: { skills: "Git" } }
  );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
universityDB> db.students.updateOne(
    { student_id: 102 },
    { $inc: { age: 1 } }
  );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
universityDB> db.students.find({}, { student_id: 1, name: 1, age: 1, gpa: 1, skills: 1, _id: 0 });
[
  {
    student_id: 101,
    name: 'Alice Smith',
    age: 21,
    gpa: 3.9,
    skills: [ 'Python', 'MongoDB', 'Git' ]
  },
  {
    student_id: 102,
    name: 'Bob Jones',
    age: 23,
    gpa: 3.4,
    skills: [ 'R', 'Python', 'SQL' ]
  },
  {
    student_id: 103,
    name: 'Charlie Brown',
    age: 21,
    gpa: 2.8,
    skills: [ 'Java', 'C++', 'Git' ]
  },
  {
    student_id: 104,
    name: 'Diana Prince',
    age: 23,
    gpa: 3.9,
    skills: [ 'Linux', 'Python', 'Networking' ]
  }
]
universityDB>
```

*Figure 6: updateOne() with $set, updateMany() with $push, updateOne() with $inc and the updated documents*

### Indexing

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.createIndex({ student_id: 1 }, { unique: true });
student_id_1
universityDB> db.students.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { student_id: 1 }, name: 'student_id_1', unique: true }
]
universityDB>
```

*Figure 7: Unique index on student_id created and listed with getIndexes()*

### Delete operations and cleanup

```
mongosh mongodb://127.0.0.1:27055 — Ex. 10

universityDB> db.students.deleteOne({ student_id: 103 });
{ acknowledged: true, deletedCount: 1 }
universityDB> db.students.deleteMany({ gpa: { $lt: 3.0 } });
{ acknowledged: true, deletedCount: 0 }
universityDB> db.students.find({}, { student_id: 1, name: 1, gpa: 1, _id: 0 });
[
  { student_id: 101, name: 'Alice Smith', gpa: 3.9 },
  { student_id: 102, name: 'Bob Jones', gpa: 3.4 },
  { student_id: 104, name: 'Diana Prince', gpa: 3.9 }
]
universityDB> db.students.countDocuments();
3
universityDB> db.students.drop();
true
universityDB> db.dropDatabase();
{ ok: 1, dropped: 'universityDB' }
universityDB>
```

*Figure 8: deleteOne(), deleteMany(), the remaining documents, drop() and dropDatabase()*

## Result

The MongoDB operations were successfully executed using mongosh shell commands. A database (universityDB) and a collection (students) were created, four documents were inserted, and they were queried using filters, comparison and logical operators, projection, findOne() and countDocuments(). Documents were updated with $set, $push and $inc, a unique index was created on student_id, and documents were removed with deleteOne() and deleteMany() (3 of the 4 documents remained, since deleteMany() found no GPA below 3.0 after Charlie Brown was deleted), before the collection and database were dropped.